

PROGRAMMATION AVANCÉE

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2025/2026

VI. MULTITHREADING EN JAVA

VI. MULTITHREADING EN JAVA

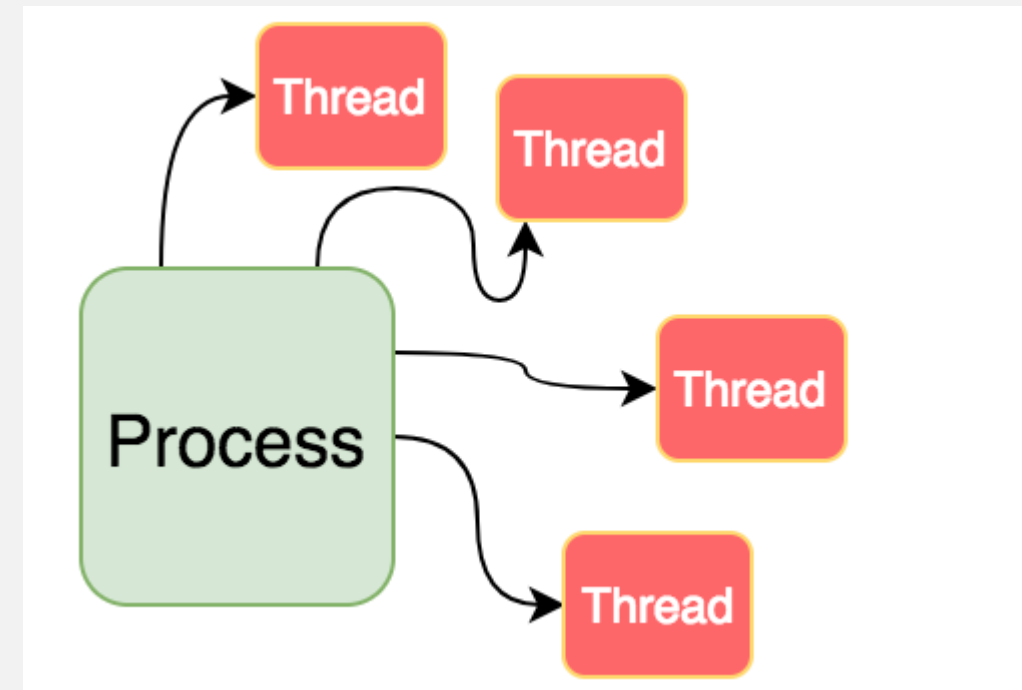
Multithreading

- Technique permettant d'exécuter plusieurs tâches (threads) simultanément dans un même programme.
- Chaque thread représente une unité d'exécution indépendante.
- Améliore performance, rapidité et réactivité.
- Géré nativement par la JVM.

VI. MULTITHREADING EN JAVA

Processus vs thread

- **Processus:**
 - Programme indépendant en cours d'exécution.
 - Possède sa propre mémoire.
 - Lourd en création et en gestion.
- **Thread:**
 - Sous-unité d'un processus.
 - Partage la mémoire et les ressources du processus.
 - Plus léger et plus rapide à créer.



VI. MULTITHREADING EN JAVA

Avantages du multithreading

1. Parallélisme:

- Exécution simultanée de plusieurs tâches.
- Permet de diminuer les temps de traitement.
- Exploite mieux les machines multi-cœurs.

2. Réactivité:

- L'application reste fluide même si une tâche est longue. Exemple : interface utilisateur + traitement en arrière-plan.

3. Exploitation des multi-cœurs

- Répartit les tâches sur plusieurs cœurs CPU.
- Améliore les performances globales.

VI. MULTITHREADING EN JAVA

Problèmes lié au multithreading

- **Conditions de course (Race Conditions):**
 - Deux threads accèdent à la même ressource en même temps.
 - Résultats imprévisibles ou incorrects.
- **Deadlocks:**
 - Deux threads se bloquent mutuellement en attendant chacun une ressource détenue par l'autre.
 - Le programme reste figé.
- **Starvation:**
 - Un thread ne reçoit jamais l'accès aux ressources.
 - Bloqué au profit d'autres threads.
 - Livelock Les threads sont actifs mais tournent en rond sans progresser.
 - L'état change continuellement mais aucune tâche ne se termine.

VI. MULTITHREADING EN JAVA

Etats d'un thread

- Voici le détail **des six états** définis par l'énumération Thread.State:

État	Description	Exemple typique
NEW	Thread créé mais pas démarré	<code>new Thread()</code>
RUNNABLE	En cours ou prêt à s'exécuter	<code>t.start()</code>
BLOCKED	Attente d'un verrou	<code>synchronized</code> concurrent
WAITING	Attente infinie d'une notification	<code>obj.wait()</code>
TIMED_WAITING	Attente avec délai	<code>Thread.sleep(1000)</code>
TERMINATED	Exécution terminée	Fin du <code>run()</code>

VI. MULTITHREADING EN JAVA

Etats d'un thread

1. **NEW:**

- Thread créé mais pas encore démarré.
- `new Thread()`
- Le code `run()` n'a pas encore commencé.

2. **RUNNABLE:**

- Thread prêt à s'exécuter ou en cours d'exécution.
- Dépend du scheduler (CPU).
- Peut alterner entre ready / running.

VI. MULTITHREADING EN JAVA

Cycle de vie d'un thread

3. **BLOCKED:**

- En attente d'un verrou (monitor lock).
- Survient lors d'un accès synchronisé occupé.

4. **WAITING:**

- Attente indéfinie d'une action d'un autre thread.
- Exemple : wait(), join() sans timeout.

5. **TIMED_WAITING:**

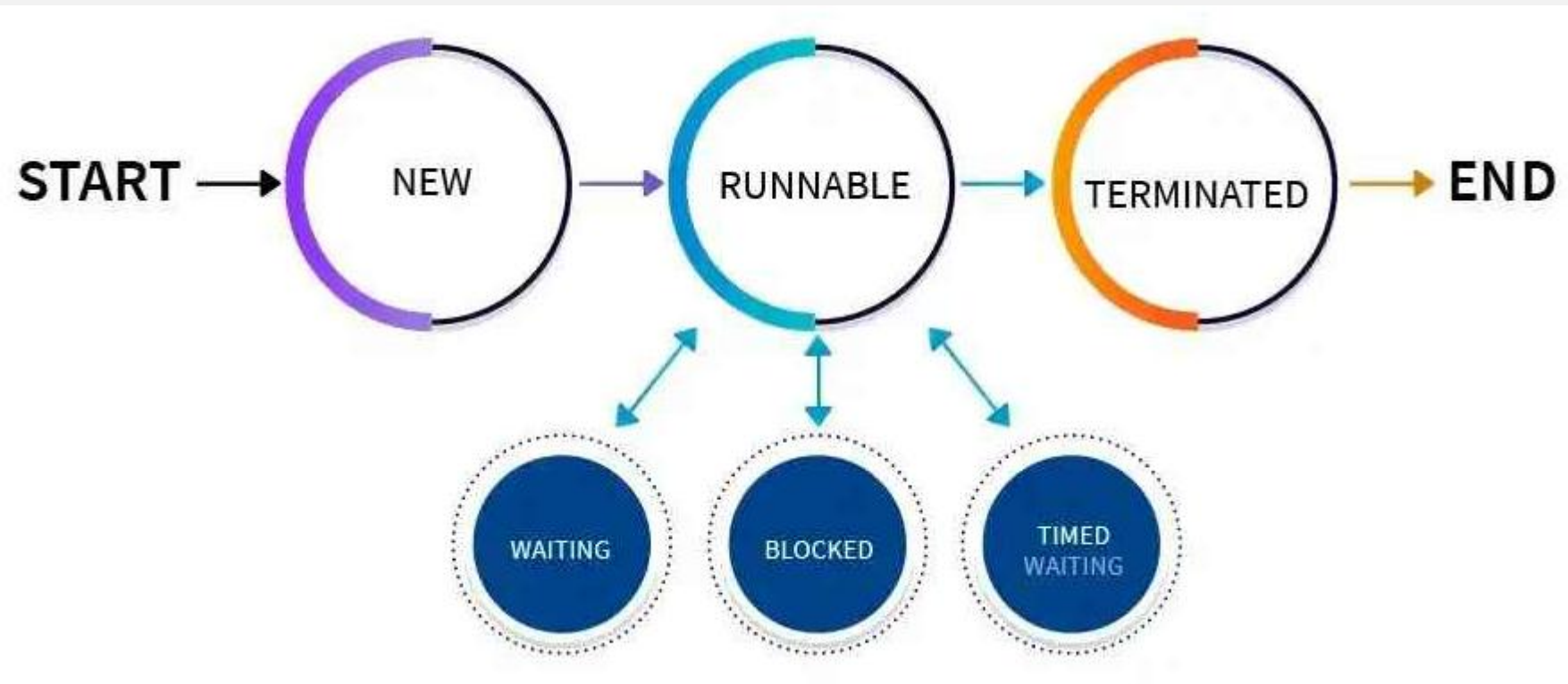
- Attente pendant une durée limitée.
- Exemple : sleep(ms), wait(ms), join(ms).

6. **TERMINATED:**

- Le thread a fini d'exécuter run() ou s'est arrêté après une exception.
- État final, non réversible.

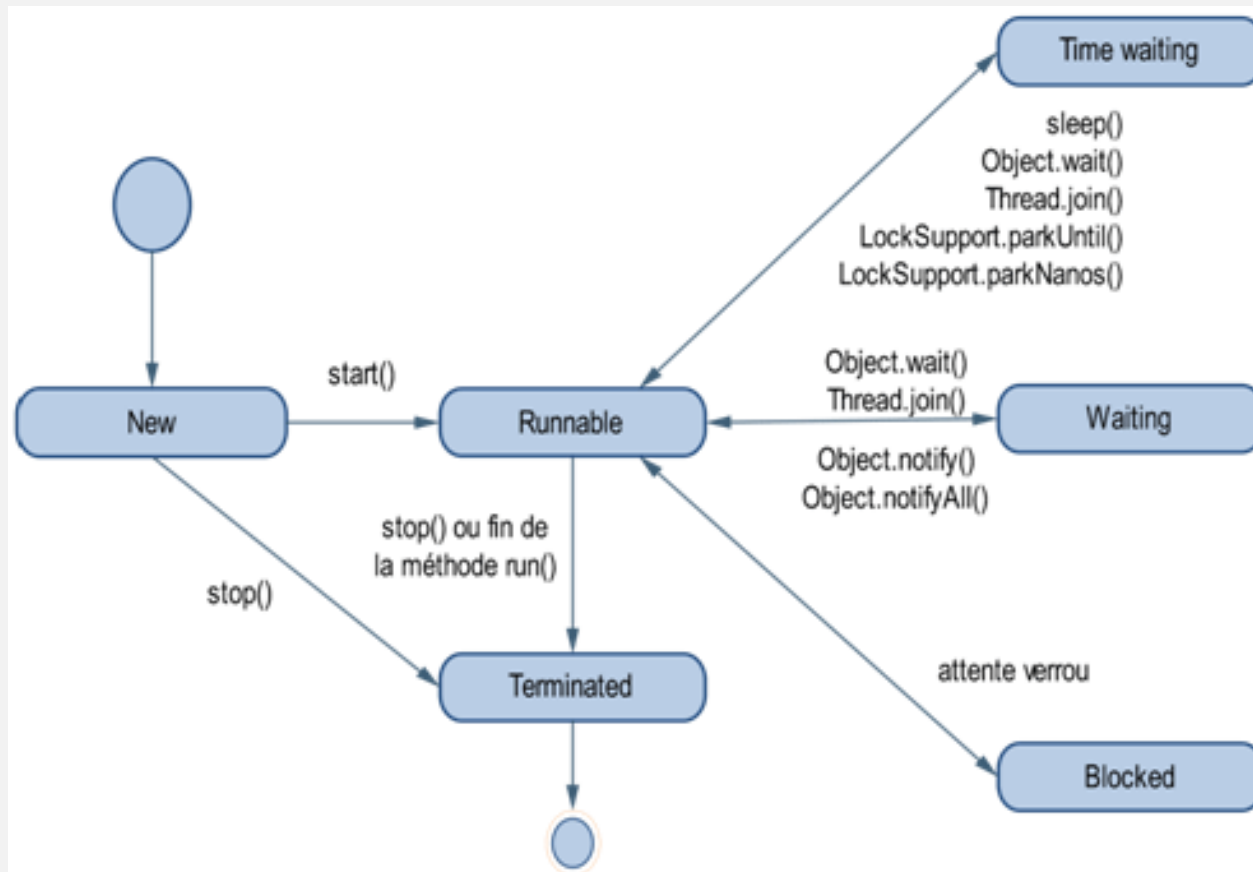
VI. MULTITHREADING EN JAVA

Cycle de vie d'un thread



VI. MULTITHREADING EN JAVA

Cycle de vie d'un thread



VI. MULTITHREADING EN JAVA

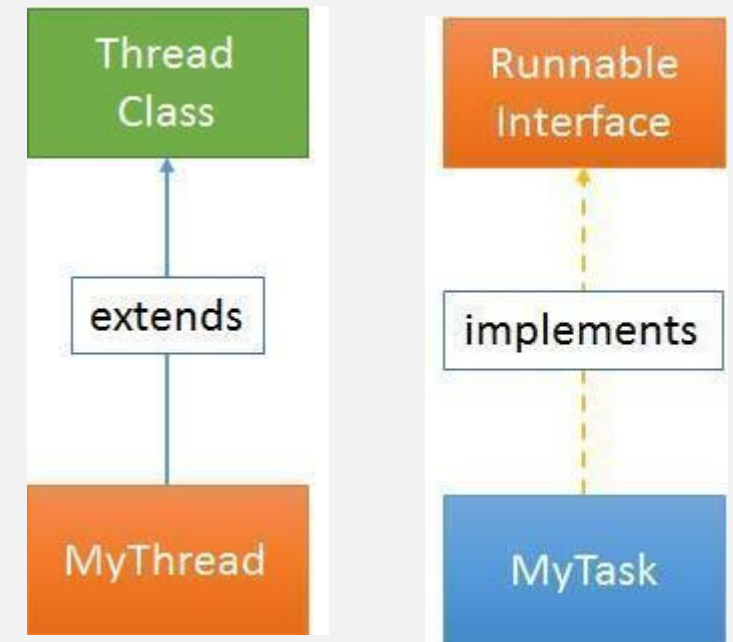
Création et lancement des threads

- Deux approches principales :
 1. Hériter de Thread
 2. Implémenter Runnable ou Callable (utiliser les lambdas (Java 8+)).
- Ces mécanismes permettent de définir une tâche qui sera exécutée en parallèle via la méthode start().

Relation entre Thread et Tâche:

Thread = exécutant

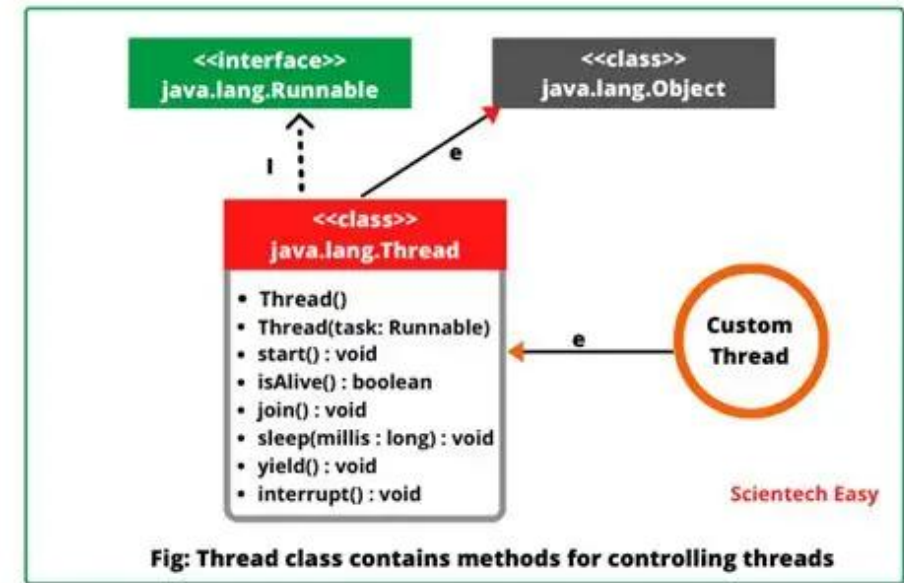
Tâche = travail à faire



VI. MULTITHREADING EN JAVA

Classe Thread: Quelques constructeurs

Constructeur	Description
<code>Thread()</code>	Crée un thread sans tâche associée (Runnable)
<code>Thread(String name)</code>	Crée un thread sans tâche mais avec un nom
<code>Thread(Runnable target)</code>	Crée un thread avec une tâche à exécuter
<code>Thread(Runnable target, String name)</code>	Crée un thread avec une tâche et un nom personnalisé



VI. MULTITHREADING EN JAVA

Classe Thread: Méthodes d'instance

Méthode	Description	Méthode	Description
<code>start()</code>	Démarre le thread et lance son exécution	<code>setName()</code>	Définit le nom du thread
<code>run()</code>	Contient le code exécuté par le thread	<code>getPriority()</code>	Retourne la priorité du thread
<code>join()</code>	Attend la fin du thread (optionnellement avec délai)	<code>setPriority()</code>	Définit la priorité du thread
<code>sleep()</code>	Met le thread en pause pendant un certain temps	<code>getState()</code>	Retourne l'état actuel du thread (<code>Thread.State</code>)
<code>interrupt()</code>	Demande l'interruption du thread	<code>stop()</code>	Arrête brutalement un thread (Dépréciée)
<code>isAlive()</code>	Vérifie si le thread est encore actif	<code>suspend()</code>	Suspend un thread (Dépréciée)
<code>getName()</code>	Retourne le nom du thread	<code>resume()</code>	Reprend un thread suspendu (Dépréciée)

Classe Thread: Méthodes statiques

Méthode	Description
<code>currentThread()</code>	Retourne une référence vers le thread en cours d'exécution
<code>sleep(long millis)</code>	Met le thread courant en pause pendant un certain temps
<code>yield()</code>	Suggère au planificateur de donner la main à un autre thread
<code>onSpinWait()</code>	Optimise les boucles d'attente active (introduit en Java 9)
<code>holdsLock(Object obj)</code>	Vérifie si le thread courant détient le verrou sur un objet

Situation	Solution
Classe hérite de <code>Thread</code>	<code>this.getName()</code> ou <code>this.setPriority()</code>
Classe implémente <code>Runnable</code>	<code>Thread.currentThread().getName()</code>
Code générique / librairie	<code>Thread.currentThread()</code> pour identifier le thread courant

VI. MULTITHREADING EN JAVA

Tâche: Runnable

- Une tâche est le travail à effectuer par un thread.
- En Java, c'est généralement représenté par :
 - une classe qui implémente **Runnable**
 - une classe qui implémente **Callable** (si tu veux un retour de résultat)

```
@FunctionalInterface  
public interface Runnable {  
    void run();  
}
```

```
@FunctionalInterface  
public interface Callable<V> {  
    V call() throws Exception;  
}
```


VI. MULTITHREADING EN JAVA

Race conditions

- **Définition** : Situation où plusieurs threads accèdent et modifient simultanément une même ressource, et où le résultat dépend de l'ordre d'exécution.
- **Conséquence** : Résultat imprévisible et bugs difficiles à détecter.
- **Solutions** :
 1. synchronized / bloc de synchronisation
 2. Objets atomiques (AtomicInteger)
 3. Verrous explicites (ReentrantLock)

Problème: Race conditions

```
public class SharedValue {  
    private int value;  
  
    public SharedValue(int value) {  
        this.value = value;  
    }  
  
    public void increment() {  
        value++;  
        System.out.println(Thread.currentThread().getName()+" "+value);  
    }  
}
```

```
public class TestSynchronized {  
  
    public static void main(String[] args) throws  
        InterruptedException {  
        SharedValue s=new SharedValue(5);  
        Thread t1=new Thread(()->s.increment(), "thread1");  
        Thread t2=new Thread(()->s.increment(), "thread2");  
        t1.start();  
        t2.start();  
    }  
}
```

thread2 6
thread1 7

thread2 7
thread1 7

thread1 7
thread2 7

- `x++` : ce n'est pas atomique.
 1. Elle fait en réalité 3 étapes :
 2. Lire la valeur actuelle de `x`
 3. Ajouter 1
 4. Écrire la nouvelle valeur dans `x`
- Si deux threads font `x++` en même temps, leurs étapes peuvent s'entrelacer, et un incrément peut se perdre.

THREAD 1	THREAD 2
-----	-----
Étape 1 : Lit la valeur partagée <code>X = 5</code> (depuis la mémoire)	Étape 1 : Lit la valeur partagée <code>X = 5</code> (depuis la mémoire)
Étape 2 : Incrémente (<code>X + 1 = 6</code>) dans son registre CPU	Étape 2 : Incrémente (<code>X + 1 = 6</code>) dans son registre CPU
Étape 3 : Écrit <code>X = 6</code> dans la mémoire	Étape 3 : Écrit <code>X = 6</code> dans la mémoire
-----	-----
RÉSULTAT FINAL : <code>X = 6</code> ❌ Au lieu de : 7	

VI. MULTITHREADING EN JAVA

synchronized / bloc de synchronisation

- **But** : Garantir que seul un thread à la fois accède à une ressource partagée.
- **Avantages** :
 - Évite les conditions de course.
 - Rend les opérations atomiques sur les données partagées.
- **À noter** :
 - Il faut garder le bloc synchronisé le plus petit possible pour conserver le parallélisme.
 - Un thread doit acquérir le verrou de l'objet pour entrer dans le bloc
 - Les autres threads attendent que le verrou soit libéré

synchronized / bloc de synchronisation

```
public class SharedValue {  
    private int value;  
  
    public SharedValue(int value) {  
        this.value = value;  
    }  
  
    public synchronized void increment() {  
        value++;  
        System.out.println(Thread.currentThread().getName()+" "+value);  
    }  
}
```

```
public class TestSynchronized {  
  
    public static void main(String[] args) throws  
        InterruptedException {  
        SharedValue s=new SharedValue(5);  
        Thread t1=new Thread(()->s.increment(), "thread1");  
        Thread t2=new Thread(()->s.increment(), "thread2");  
        t1.start();  
        t2.start();  
    }  
}
```

```
    public void increment() {  
        synchronized (this) {  
            value++;  
            System.out.println(Thread.currentThread().getName()+" "+value);  
        }  
    }  
}
```

thread1 6
thread2 7

thread2 6
thread1 7

VI. MULTITHREADING EN JAVA

Objets atomiques

- Les **objets atomiques**:
 - Classes du package *java.util.concurrent.atomic*
 - Fournissent des opérations atomiques sur des variables partagées
 - Évitent l'utilisation explicite de `synchronized`
- Une **opération atomique** est indivisible :
 - Si un thread commence l'opération, aucun autre thread ne peut l'interrompre avant qu'elle soit terminée.
 - Par exemple, la méthode `increment()` synchronisée sur un objet est atomique : un seul thread peut exécuter la méthode à la fois.

VI. MULTITHREADING EN JAVA

Objets atomiques

Objet	Type manipulé	Méthodes clés
AtomicInteger	int	incrementAndGet(), decrementAndGet(), addAndGet()
AtomicLong	long	idem que AtomicInteger
AtomicBoolean	boolean	getAndSet(), compareAndSet()
AtomicReference	Référence d'objet T	get(), set(), compareAndSet()

Objets atomiques

```
import java.util.concurrent.atomic.*;

public class SharedValueAtomic {
    private AtomicInteger value=new AtomicInteger(0);

    public SharedValueAtomic(int value) {
        this.value.set(value);
    }

    public void increment() {
        //synchronisé grace à AtomicInteger
        value.incrementAndGet();
    }

    @Override
    public String toString() {
        return "SharedValueAtomic [value=" + value +
            "]\n";
    }
}
```

```
import java.util.concurrent.atomic.*;

public class TestSynchornized {
    public static void main(String[]
args) throws InterruptedException {
        SharedValueAtomic s=new
SharedValueAtomic(5);
        Thread t1=new Thread(()-
>s.increment(), "thread1");
        Thread t2=new Thread(()-
>s.increment(), "thread2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(s);
    }
}
```

SharedValueAtomic [value=7]

VI. MULTITHREADING EN JAVA

Verrous explicites (**ReentrantLock**)

- **ReentrantLock** est:
 - Classe du package `java.util.concurrent.locks`
 - Permet de gérer les verrous de manière plus flexible que `synchronized`
 - Offre des fonctionnalités avancées : verrouillage conditionnel, tentative de verrouillage avec timeout, verrouillage réentrant.
- **Avantages par rapport à `synchronized`:**
 - Possibilité de tester si le verrou est disponible (`tryLock()`)
 - Support du timeout (`tryLock(timeout, unit)`)
 - Verrouillage réentrant (un thread peut reprendre le même verrou plusieurs fois)
 - Gestion plus fine avec Conditions (équivalent avancé de `wait/notify`)

```

import java.util.concurrent.locks.*;

public class SharedValueLock {
    private int value;
    ReentrantLock lock=new ReentrantLock();

    public SharedValueLock(int value) {
        this.value = value;
    }

    public void increment() {
        lock.lock();
        try{
            value++;
            System.out.println(Thread.currentThread().getName()+" "+value);
        }finally {
            lock.unlock(); // garanti d'être exécuté
        }
    }

    public String toString() {
        return "SharedValueLock [value=" + value + "]";
    }
}

```

```

import java.util.concurrent.atomic.*;

public class TestSynchornized {

    public static void main(String[] args)
    throws InterruptedException {
        SharedValueLock s=new SharedValueLock(5);
        Thread t1=new Thread(()->s.increment(), "thread1");
        Thread t2=new Thread(()->s.increment(), "thread2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(s);
    }
}

```

```

thread2 6
thread1 7
SharedValueLock [value=7]

```